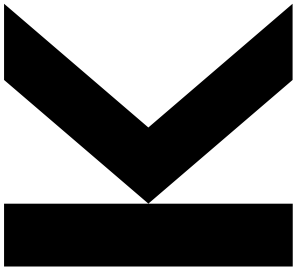
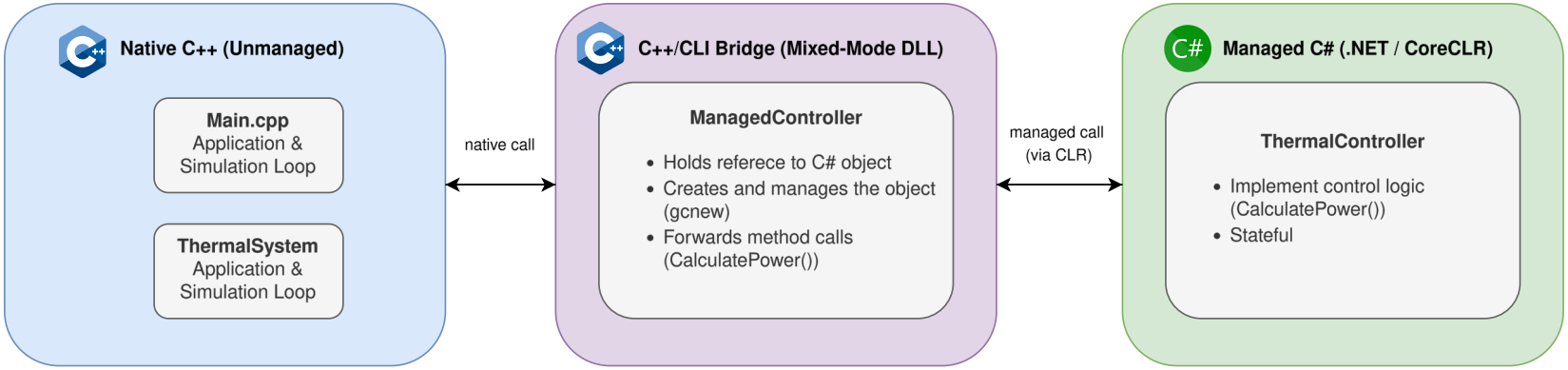
REPLACING C++/CLI IN THE KULI CALCULATION KERNEL



Proof of Concept — Managed Access: hostfxr vs. Native AOT

JKU

C++/CLI - DEMO ARCHITECTURE

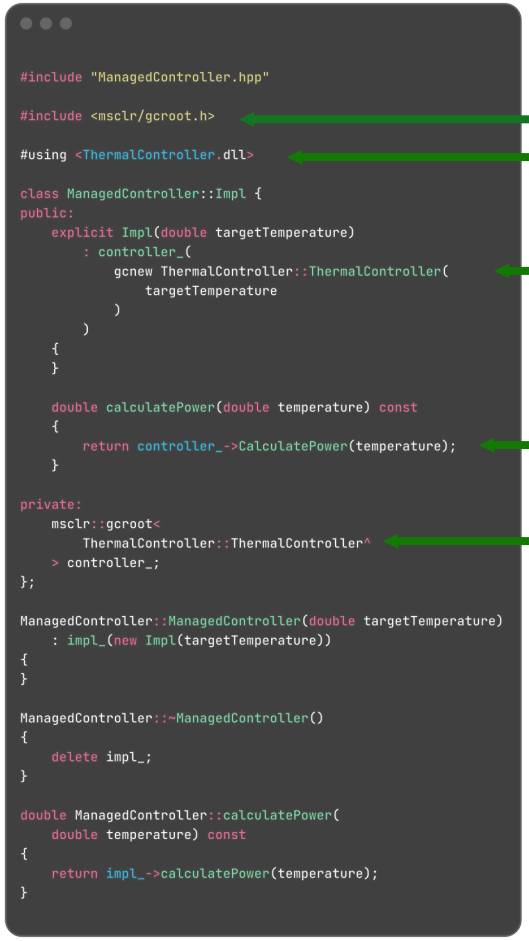


Key characteristics

- C++/CLI acts as a bridge between native C++ and managed C#
- Direct, OO access to the C# controller
- Runs in the CoreCLR runtime
- Windows only

JKU

C++/CLI – MANAGEDCONTROLLER.CPP



```cpp
#include "ManagedController.hpp"

#include <msclr/gcroot.h>

#using <ThermalController.dll>

class ManagedController::Impl {
public:
    explicit Impl(double targetTemperature)
        : controller_(
            gcnew ThermalController::ThermalController(
                targetTemperature
            )
        )
    {
    }

    double calculatePower(double temperature) const
    {
        return controller_->CalculatePower(temperature);
    }

private:
    msclr::gcroot<
        ThermalController::ThermalController^
    > controller_;
};

ManagedController::ManagedController(double targetTemperature)
    : impl_(new Impl(targetTemperature))
{
}

ManagedController::~ManagedController()
{
    delete impl_;
}

double ManagedController::calculatePower(
    double temperature) const
{
    return impl_->calculatePower(temperature);
}
```

Allows a native C++ class to safely hold a reference to a managed .NET Object

Loads the C# Assembly

gcnew creates an object on the .NET managed heap and returns a tracking handle

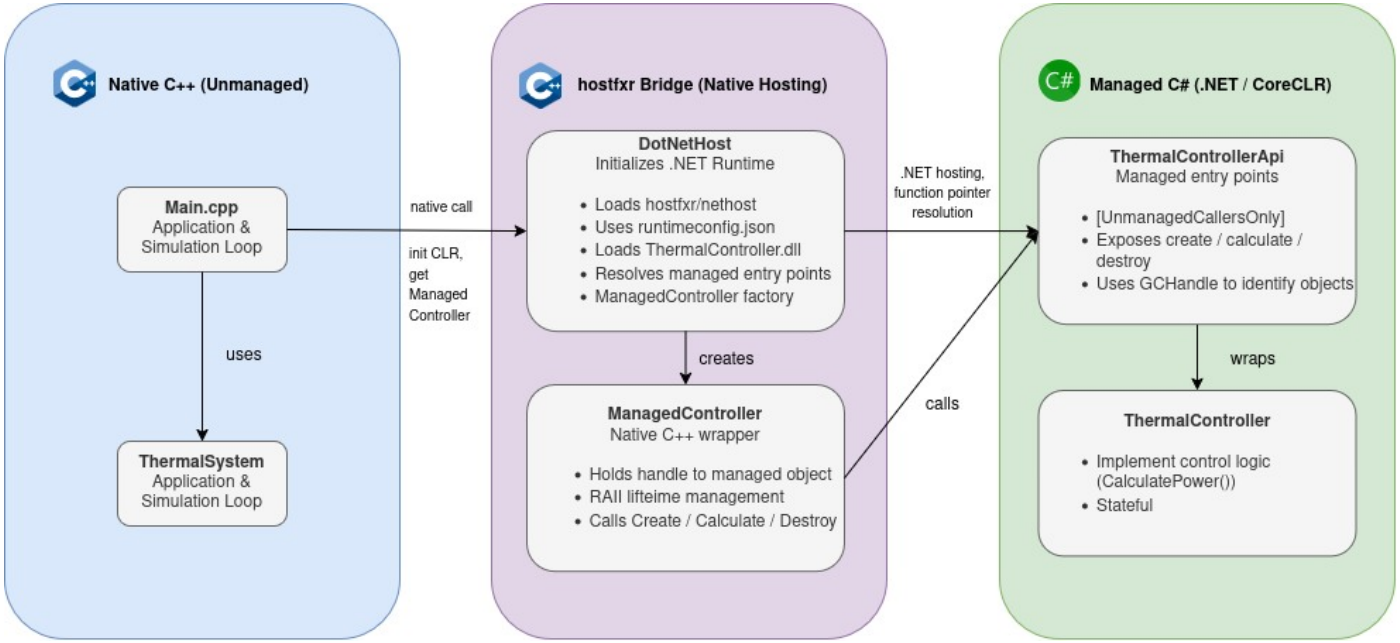
Call a C# function via handle

Tracking handle, needed because GC may rellocate objects

Wrapper around impl, to hide C++/CLI implementation from native C++

JKU

APPROACH A — HOSTFXR + REVERSE P/INVOKE

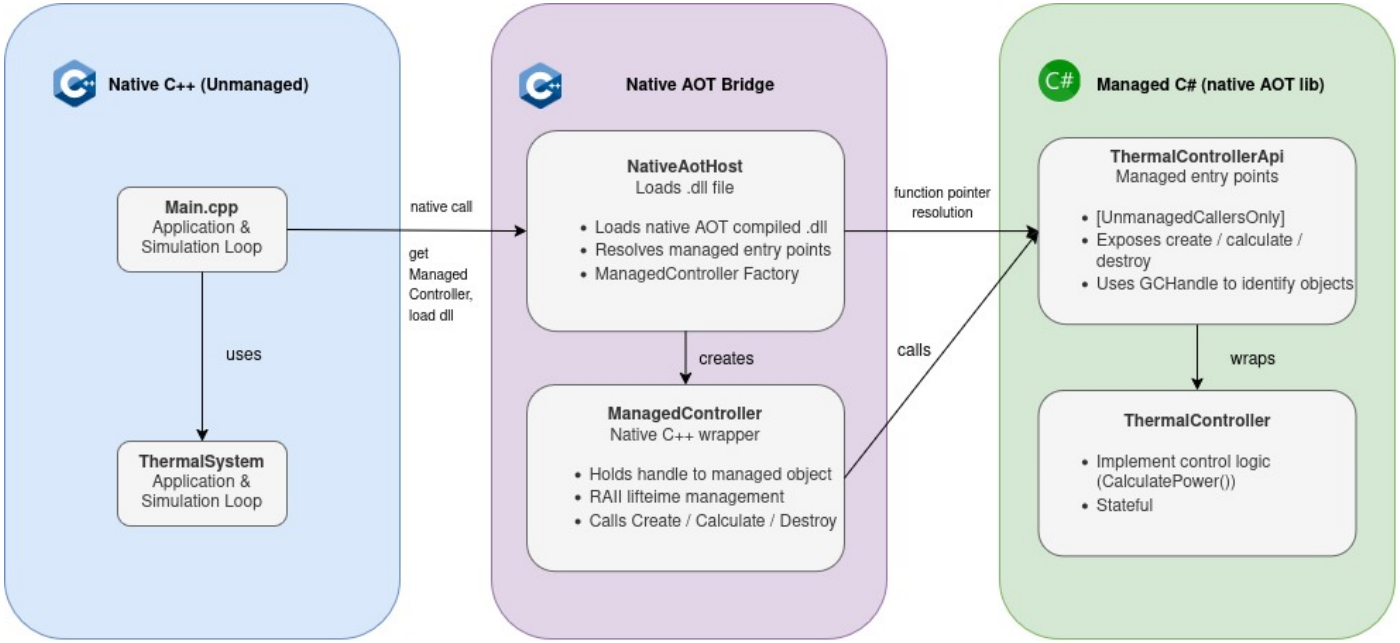


Key characteristics

- Native C++ hosts CoreCLR using hostfxr
- Uses regular managed .NET assembly and runtimeconfig.json
- Works well with existing .NET code, but requires boilerplate
- Cross-platform

JKU

APPROACH B — NATIVE AOT (AHEAD OF TIME COMPILATION)

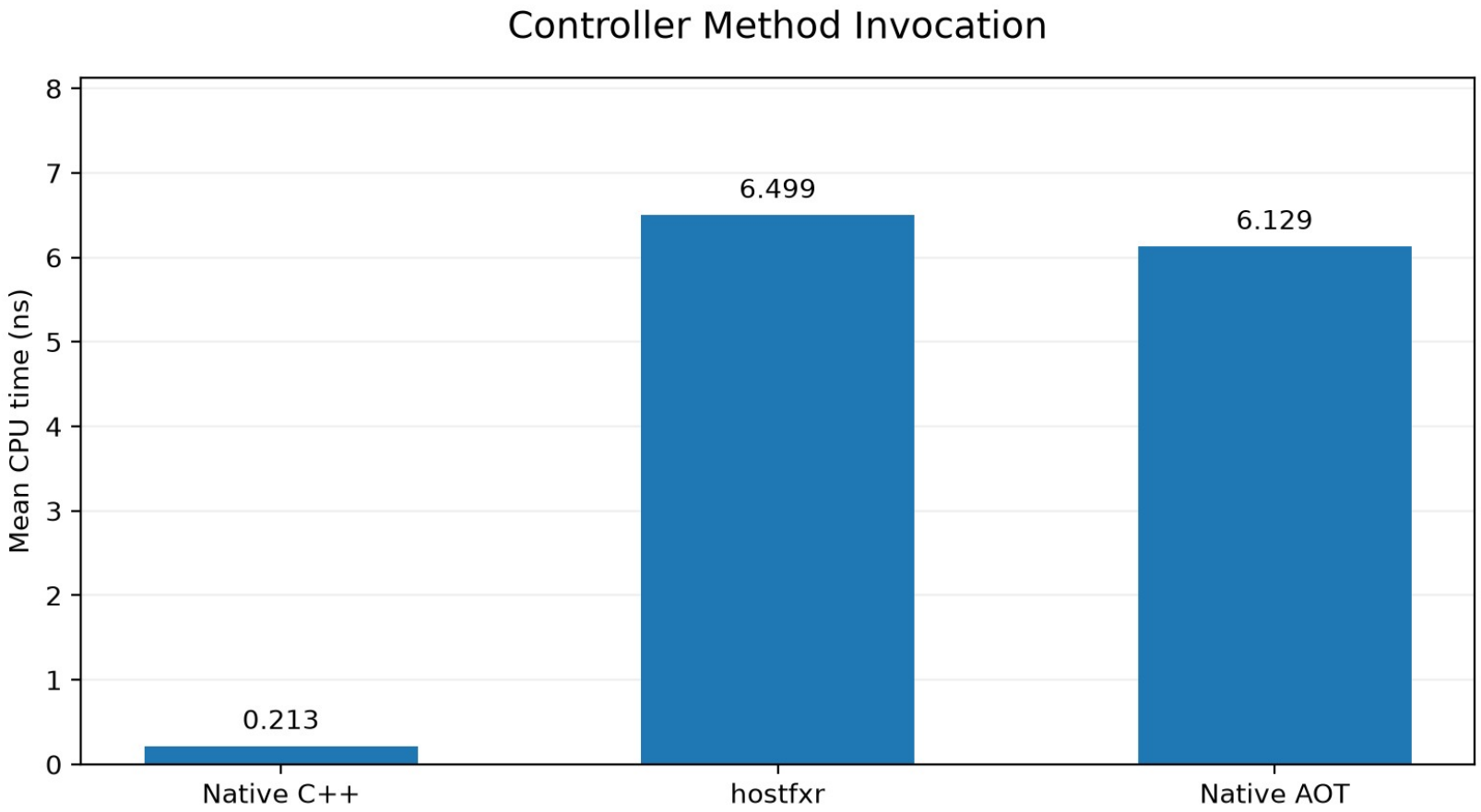


Key characteristics

- C# code is compiled ahead of timme to a native library (.so / .dll)
- Native C++ loads the library at runtime and calls its exported C ABI
- No CoreCLR hosting or runtime installation required
- .NET feature limitations (reflection, dynamic loading)
- Cross-platform

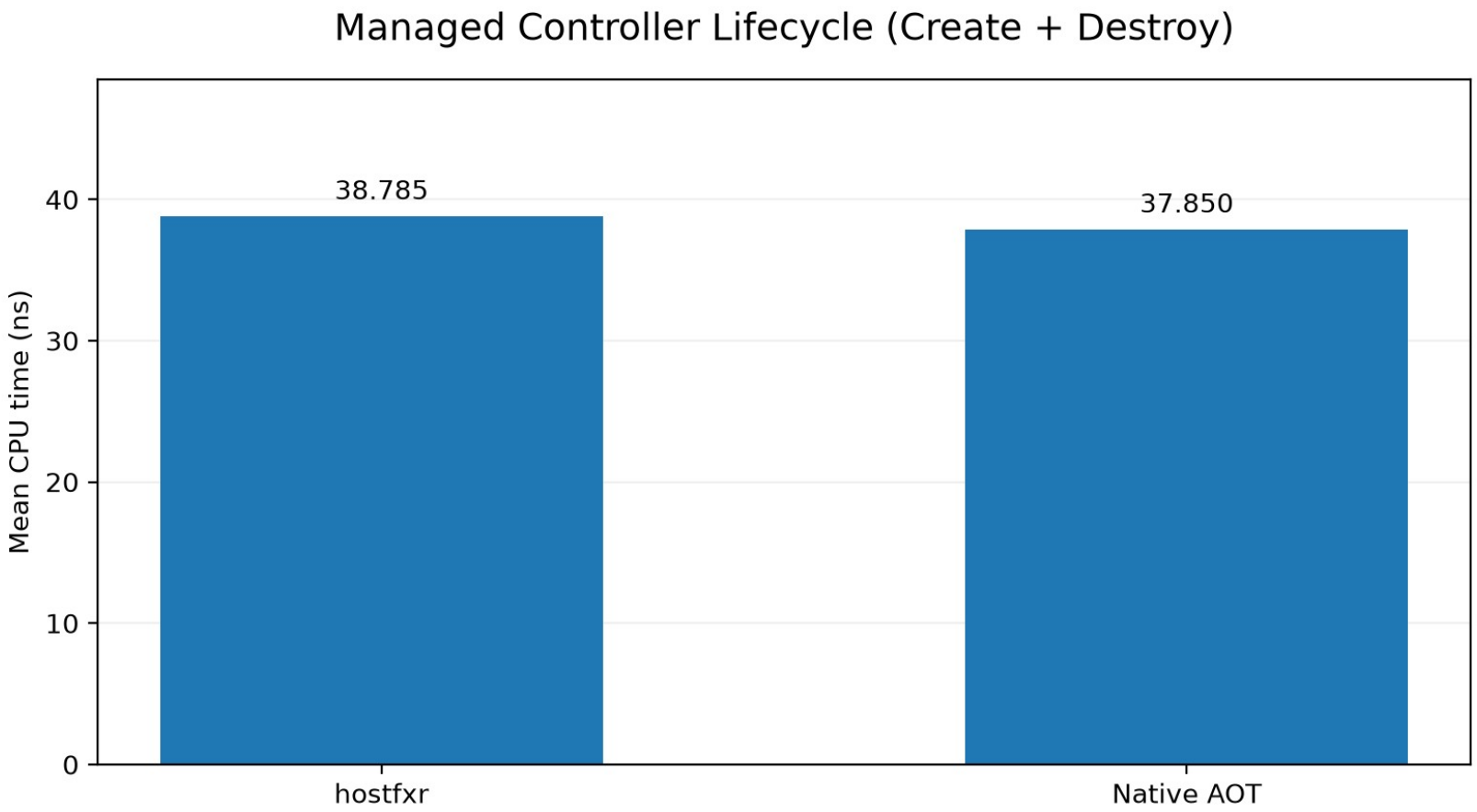
JKU

PRELIMARY PERFORMANCE RESULTS



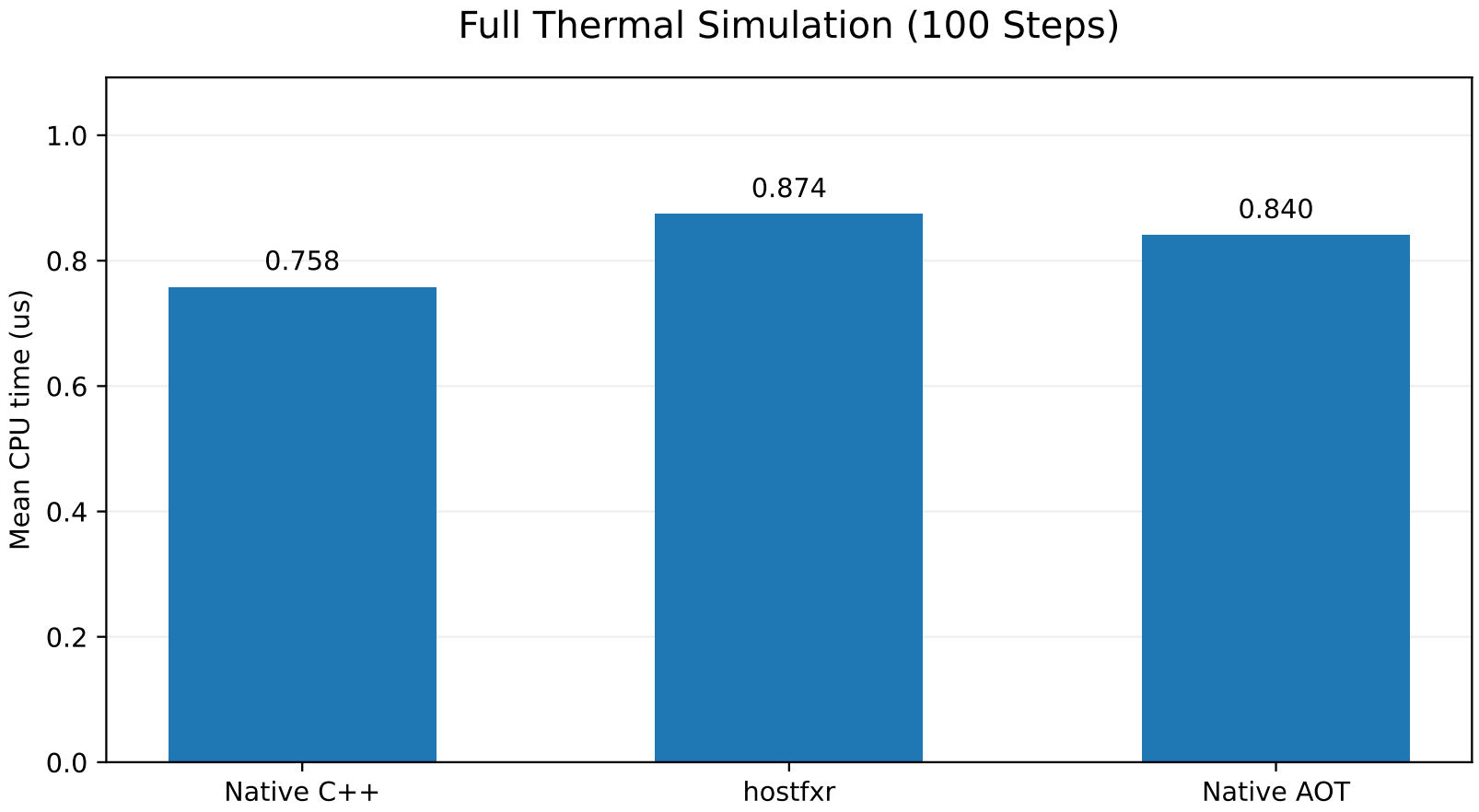
JKU

PRELIMARY PERFORMANCE RESULTS



JKU

PRELIMARY PERFORMANCE RESULTS



JKU

EVALUATION: STRENGTHS & LIMITATIONS

Criterion	hostfxr	Native AOT
Linux support	yes	yes
.NET compatibility	Broad CoreCLR support	AOT restrictions
Runtime	Requires .NET runtime	Self-contained native lib
Interop Complexity	Hosting + func pointer	Native exports + func pointers
Primitive types	direct	direct
Structs	Direct if blittable and ABI compatible	Direct if blittable and ABI compatible
Arrays	Pointers + lengths	Pointer + lengths
Managed objects	handles	handles
Exception handling	Explicit error translation	Explicit error translation

Blittable = same memory layout in Managed C# and Unmanaged C++ code

FURTHER INTEROPERABILITY TECHNOLOGIES TO EVALUATE

Key question: Can migration effort be reduced through generated bindings, or does KULI require a different interoperability architecture?

- SWIG + Directors
- CPPSharp
- Custom Wraper generation
- gRPC